beamfeat: beam-search feature construction with false-discovery-rate controlled selection

Olanrewaju M. Daramola
ORCID: 0009-0006-3327-2047
Independent Researcher

28 July 2026

Abstract

Given a table of measurements, `beamfeat` searches for short mathematical formulas, such as $(x_0/x_2) \cdot x_1$ or $\log(a)\log(b)$, that explain a quantity of interest, and then applies a statistical test to decide which of the candidate formulas reflect real structure rather than coincidence. The result is a model a person can read as an equation, together with an explicit statement of how much statistical confidence it carries. Automated formula search evaluates thousands of candidates against the same data, so some will fit by chance; `beamfeat` controls the expected fraction of such spurious selections (the false discovery rate, FDR) and reports honestly, through a fitted flag and visible warnings, whenever that guarantee cannot be given. It is packaged as scikit-learn (1) estimators over numpy (2) and scipy (3), passing scikit-learn's own compatibility checks, with optional dimensional analysis that rejects physically meaningless expressions (metres plus kilograms) before any numerical work.

Introduction

Researchers who need interpretable models from tabular data, whether recovering an empirical law, screening engineered features for a regression, or auditing which constructed variables genuinely relate to an outcome, currently choose between tools that construct features without error control and tools that control errors without constructing features. The selection step is the scientific crux: candidates are generated adaptively and tested on the same data, a selective-inference setting (4; 5): the data that suggested each hypothesis also tests it, so naive p-values are optimistically biased. `beamfeat` treats that step as the inference problem it is. It provides an exact permutation test of marginal association, each candidate's relationship to the target considered alone (6), with Benjamini–Hochberg (7) or Benjamini–Yekutieli (8) correction, the knockoff filter in both its fixed-X (9) and model-X (10) forms, routed by sample size to the regime where each construction's assumptions hold, and a default data split (search on one half of the training rows, selection tested on the other) in the sense of Cox (11), which restores the premise the guarantees require: that candidates were fixed before seeing the data on which they are tested.

Related work

`autofeat` (12) is the closest predecessor: compact symbolic features feeding a linear model, with exhaustive expansion and an L1-based selection thresholded against injected noise columns: effective, but with no stated error-rate guarantee. `OpenFE` (13) generates features for gradient-boosted models and ranks them by boosting-based importance; Deep Feature Synthesis (14) targets relational data; the reference knockoffs implementation `knockpy` (9; 10; 15) selects among *given* features and constructs nothing, so its row in the benchmark table below isolates the contribution of construction itself. Symbolic regressors such as `PySR` (16)

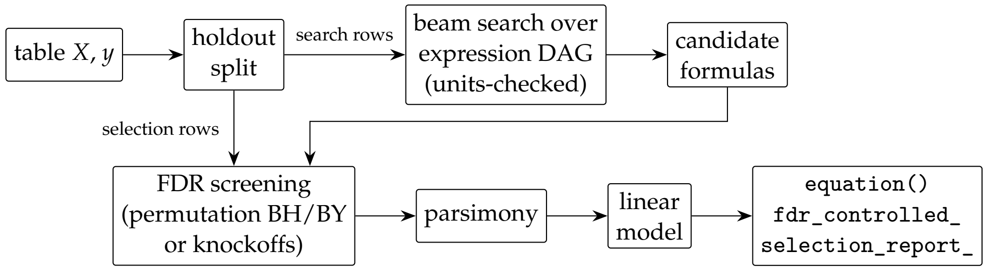

Figure 1: The pipeline: candidates are constructed on one part of the data and tested on the other, so adaptive search cannot bias the selection p-values; only screened survivors reach the fitted model.

conduct a strictly more powerful expression search, fitting constants inside nonlinearities, with no error control. The closest design precedent is `tsfresh` (17), which filters mass-generated time-series features under Benjamini–Yekutieli control; `beamfeat` brings that generate-then-error-controlled-filter design to symbolic expressions on tabular columns.

Contributing the guarantee to an existing tool was not viable: it requires a different selection statistic (exact under permutation), a holdout architecture threaded through the fit path, and an expression engine whose numerical failures are recorded rather than masked: a redesign of each tool's core rather than a patch. The practical case is also material: during benchmarking, neither `autofeat` 2.1.3 nor `OpenFE` 0.0.12 ran on a current stack without intervention. `autofeat` raises `TypeError: check_array() got an unexpected keyword argument 'force_all_finite'` whenever it transforms or predicts on unseen data having engineered at least one feature, on scikit-learn ≥ 1.8 (the argument was renamed in 1.6 and removed in 1.8), that is, in its normal supervised use, and its pins (`numpy<2.0`, `pandas<3.0`) downgrade current environments. `OpenFE` calls `mean_squared_error` with the `squared` argument removed in scikit-learn 1.6. Both were therefore run in isolated environments, with the `OpenFE` compatibility patch disclosed in the repository's benchmark code, while `beamfeat`'s tests run unmodified from scikit-learn 1.6 through 1.9 with no upper version pins.

Method

Three trade-offs define the design. First, the selection statistic is *marginal* association, absolute Pearson correlation for regression and eta-squared (the correlation ratio for class labels) for classification: a fixed function of the data, so permutation p-values are exact, where statistics that re-tune themselves on the observed target (a cross-validated lasso penalty) break the exchangeability a permutation test rests on (under the null, the target and its shuffles are statistically interchangeable; a statistic that adapts to the observed target destroys that symmetry). The price is measured precisely on Friedman #1 (18). A least-squares oracle given the ideal depth-2 basis $\{ab, (ab)^2, c, c^2, d, e\}$ reaches R^2 0.960 ± 0.003 over six draws. Marginal screening never admits c, because the centred quadratic viewed alone is nearly independent of the target; it admits c^2 on some draws and not others, since $(c-0.5)^2$ expands to carry a little marginal signal through that term. The best any model restricted to the admitted set can reach is 0.874 ± 0.006, against the pipeline's 0.776 ± 0.013. Roughly 0.086 is lost to what marginal screening cannot see and 0.098 to the search. Part of that remaining gap is a search limitation that persisted under wider beams, greater candidate diversity, and lookahead scoring; closing it fully would require scoring candidates as a set rather than one at a time, which reintroduces exactly the pick-what-fits-the-noise behaviour the design excludes. Heuristic tools that select jointly outperform on this structure while certifying nothing.

Second, correctness defaults are chosen over power and convenience. Benjamini–Yekutieli is the estimator default because it is valid under arbitrary dependence, which is the regime engineered candidates occupy: they share parents and are mutually correlated by construction, so the positive dependence Benjamini–Hochberg assumes cannot be taken for granted. Over 100 pure-noise pipelines, where every selection is by

Table 1: Benchmark results from the repository's benchmark code. `OpenFE`'s recovery figure is over five of the six stress datasets; its core-suite rows are not applicable, since it targets gradient-boosted lift rather than formulas. It ran only on the stress suite: its recovery and false-feature entries cover the five stress problems that declare generating columns, and its fit time averages all six, so that cell is not directly comparable to the core-suite times in the other columns.

measure	beamfeat	autofeat	OpenFE	knockpy	LightGBM	ridge
generating columns recovered	**10/10**	8/10	0/5	0/10	—	—
mean R^2, core suite	**0.9989**	0.9968	—	0.8450	0.9798	0.8442
stress sets with false features	**0 of 5**	3 of 5	0 of 5	1 of 5	—	—
mean fit time (s)	**0.2**	12	1.4	0.02	0.3	$<$0.01

definition false, BH returned features in 6 trials and BY in 1 (false discovery proportions 0.06 and 0.01). Both sit under the nominal 0.10 and the difference is not significant at this trial count ($p = 0.12$), but BH fires on noise several times as often, and it is the tail behaviour a default should be chosen for. When nothing passes selection, the default returns *no* constructed features (an intercept-only model and a visible warning) rather than unvetted search output. A parsimony step (greedy forward selection *within* the screened set) keeps fitted equations compact while the full screened set, with exact p- and q-values (19) per candidate, stays auditable; the q-level guarantee certifies the screening set, and the documentation says so. A post-fit check on the selection holdout separates two claims users conflate: FDR-vetted association and a generalising fit: a negative held-out R^2 raises a visible warning naming the gap.

Third, boundaries are stated with measurements. Piecewise targets belong to trees (LightGBM 0.998 against 0.808 on a threshold problem; `OpenFE` records 0.998 there as well, but bit-identically to raw LightGBM and from a feature touching only an irrelevant column, so the credit belongs to its gradient-boosted consumer rather than to anything it constructed); the exponential operator ships but is not a default (enabling it changed no outcome on the physics panel below, though individual scores move); overflow and domain errors exclude a candidate loudly rather than silently; and units, supplied as pint quantities or plain strings, are enforced at expression construction.

Results

The repository's significance rests on evidence that ships with the code and can be re-run, rather than on claims. End to end at nominal FDR 0.10, 200 repeated trials with a true signal produced no false discovery at all, a 95% upper bound of 0.015 on the true rate, with power 1.000 and no fit falling back to unvetted output; 60 trials with no signal selected nothing. At the selector level, realised FDR tracked the bound each correction must respect. Over 100 trials Benjamini–Hochberg realised 0.046 ± 0.008, 0.084 ± 0.012 and 0.161 ± 0.016 at nominal 0.05/0.10/0.20, against its ceiling of $q\,m_0/m$ = 0.040/0.080/0.160; Benjamini–Yekutieli, stricter by a harmonic factor, realised 0.008 ± 0.004, 0.018 ± 0.005 and 0.046 ± 0.008. Power was 1.00 throughout. The bounds are derived from the design rather than asserted, and `benchmarks/selector_calibration.py` fails if a realised rate sits *significantly* above its bound, that is, if the bound falls below the whole 95% interval; a point estimate a little above its bound, as all three Benjamini–Hochberg rates are here, is ordinary Monte Carlo error.

On a twelve-equation physics panel under the symbolic-regression community's criterion ($R^2 > 0.999$ at 0.1% noise) (20; 21), `beamfeat` solves 10/12 and recovers 8/12 in *exact symbolic form*, checked by algebraic proportionality, a test a merely close numeric fit cannot pass, at roughly 0.6 s per equation; each miss marks a named boundary (a depth-3 rational, a literal constant, a depth-4 nesting, the Gaussian's exponential).

Head-to-head, using the benchmark code shipped in the repository (identical data splits; each compared tool run in an environment matching its own declared dependencies):

Two ten-problem sets underlie Table 1 and they are not the same ten. Recovery asks whether some returned feature references all of the generating columns, over the ten synthetic problems that declare them; the mean

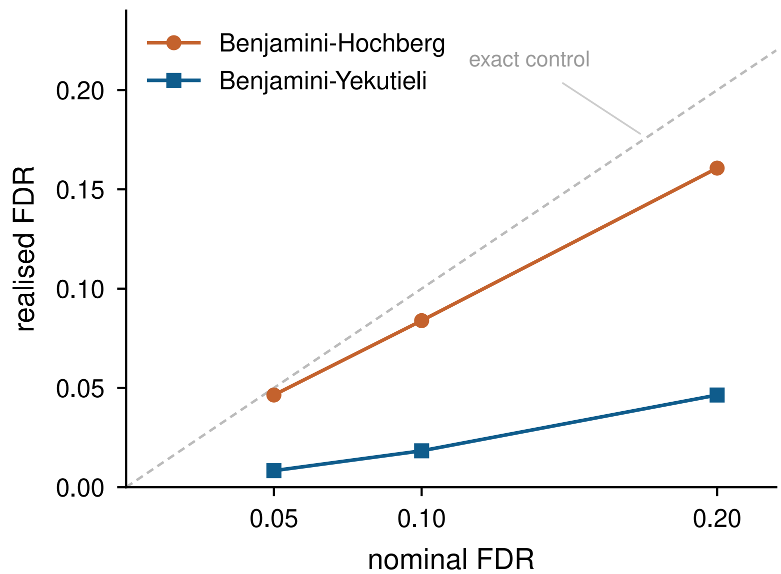

Figure 2: Realised against nominal false discovery rate for the permutation selector, 100 trials on Gaussian designs with 25 candidates of which 5 carry signal. Benjamini–Hochberg tracks its ceiling of $q\,m_0/m$; Benjamini–Yekutieli, stricter by a harmonic factor, sits well below nominal. Power was 1.00 at every level. Regenerated by `benchmarks/selector_calibration.py`, which derives each bound from the design and fails if the bound falls below a realised rate's 95% interval.

R^2 column covers the ten formula-recovery problems of the core suite, which substitutes `purely_linear` for `heavy_tail_t2`. Fit times are indicative figures from one machine (Dell Inspiron 16 Plus 7640, 22 logical cores, Linux 7.0, Python 3.11.15, scikit-learn 1.7.2, numpy 1.26.4, no accelerator), the pinned environment the compared tools require; the cross-method ratios, measured on identical hardware, are the stable quantity.

Under Student-$t(2)$ noise the exact formula is recovered at R^2 0.956 (LightGBM 0.866): the permutation test is distribution-free (it assumes nothing about the noise's shape). Across seven real datasets (22), `beamfeat` leads on two: mpg (0.871 against LightGBM's 0.853) and tips, where gradient boosting overfits below ridge while the FDR gate holds; it matches ridge on diamonds from a single interpretable feature, and trails LightGBM on penguins and breast cancer and ridge on diabetes. The widest margin against it is California housing, at 0.700 against LightGBM's 0.843, where the signal is spatial rather than algebraic and there is no compact expression to find; it still clears the ridge anchor there by 0.107. A feature constructor should concede when there is nothing to construct, and the honesty flag does so on every fit.

A separate 360-fit study, included in the repository (nine datasets (23; 24; 25), eight methods, five splits, analysed by average ranks and a Friedman test (26)) places these results in a wider field. One of the nine is the Boston housing set, which scikit-learn removed as ethically contested; it is retained here deliberately, because the `autofeat` authors report their own Boston failures (12) and the comparison is therefore direct. Every feature-construction tool there hands its features to the same `RidgeCV`, so that comparison isolates the constructed features rather than the estimator; `beamfeat` appears twice, once as the shipped estimator with its own internal ridge and once as a transformer feeding the shared model. The two agree to a median absolute difference of 0.0003 across 45 paired fits, with a largest discrepancy of 0.016, so the result comes from the features and not from the downstream model.

Mean held-out R^2 was 0.803 for `beamfeat` against 0.810 for a random forest (27) and 0.798 for LightGBM (28) on raw features, 0.736 for `OpenFE`, 0.704 for ridge, -1.56 for `autofeat` and -2.48 for `featuretools`. `beamfeat` took the best average rank (3.22 of seven methods, the tree baselines 3.44, `autofeat` 3.67), though with nine datasets the omnibus test is underpowered and does not separate them ($\chi^2 = 6.71$, $p = 0.35$). The paired tests are sharper: `beamfeat` beat `OpenFE` ($p < 0.0001$), `featuretools` and ridge ($p < 0.0001$) under

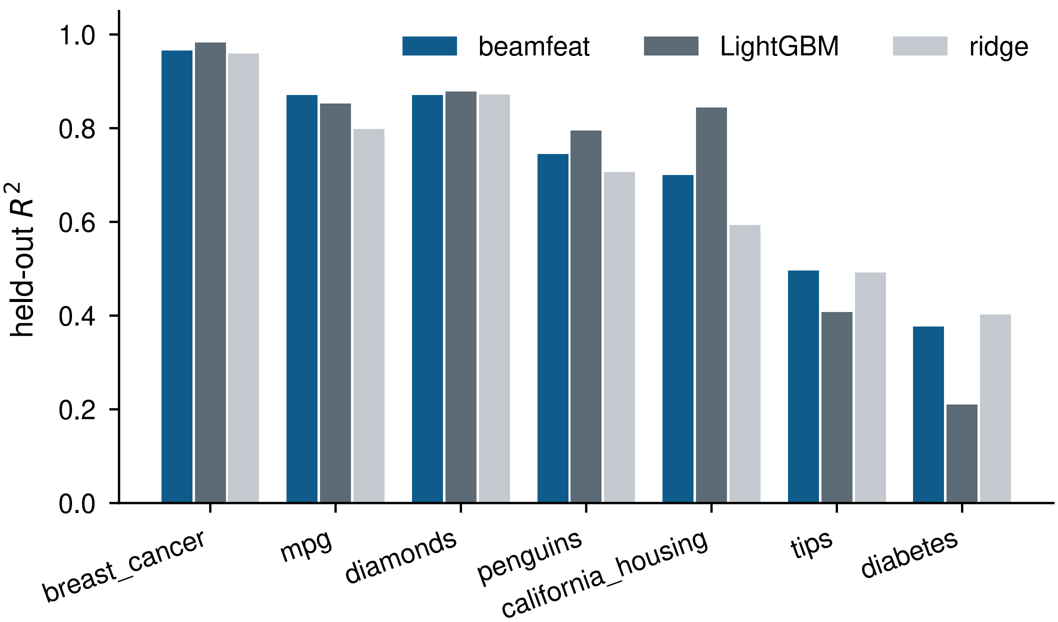

Figure 3: Held-out R^2 on seven real datasets, numeric columns only. `beamfeat` leads on mpg and tips, matches ridge on diamonds from a single constructed feature, and trails the gradient-boosted baseline where the signal is not algebraic. California housing is the widest margin against it: the structure there is spatial, and there is no compact expression to find.

the shared downstream model, a protocol that understates `OpenFE` in its gradient-boosted home setting. It was indistinguishable from the random forest ($p = 0.60$) and LightGBM ($p = 0.86$), and the comparison against `autofeat` sits at the margin ($p = 0.059$) on a median difference of essentially zero: the two are level on a typical split, and the mean gap of $+2.36$ comes entirely from the splits where `autofeat` fails outright.

The worst-case column is the more informative one. `beamfeat`'s poorest single fit of 45 was R^2 0.355 and none fell below zero, against -103.2 for `autofeat` and -57.3 for `featuretools`, which returned two and six fits respectively worse than predicting the target mean. `beamfeat` reached that from 9.2 constructed features on average, `OpenFE` from 7.3, `autofeat` from 16.8 and `featuretools` from 104.7; measured against ridge on the unengineered columns, `autofeat` and `featuretools` both ended below the level of constructing nothing at all. `beamfeat` was roughly 60 times faster than `autofeat`, and the FDR guarantee held on all 45 fits.

That study also records a reproducibility asymmetry: `beamfeat` returns bit-identical results across runs given a seed: a determinism test in the suite refits and compares formulas and predictions. `autofeat` could not be made reproducible from outside. It exposes no `random_state`; its noise-injection feature screen draws decoy features through the global NumPy generator, and the one seed it sets internally (`np.random.seed(i)` per selection run) overrides any seed a caller provides. Six runs on one identical split of Friedman #1, at the library's default settings in the pinned environment, returned R^2 from $+0.952$ to -109.8 and between 17 and 26 selected features; pinning `OMP_NUM_THREADS`, `NUMBA_NUM_THREADS` and `MKL_NUM_THREADS` to 1 changed nothing. The variation appears where selection is marginal, which is where it matters: on problems with an unambiguous signal repeated processes agree exactly. It is visible at the level of the whole study too: four executions of the nine-dataset comparison returned `autofeat` mean R^2 of 0.746, -1.69, 0.754 and -1.56, with worst single fits from -2.28 to -108.9. Any single `autofeat` figure, including those above, is one draw from that distribution. Catastrophic held-out scores are a documented property of the method rather than an artifact of this harness: Horn et al. (12) report test R^2 of -12.4 on diabetes and 0.048 on Boston at three feature-engineering steps, attributing it to tens of thousands of features fitted against fewer than 500 rows. This study uses two steps, the library default, at which their Table 2 reports no such failure. Further reproducibility signals: 370 automated tests including scikit-learn's own `check_estimator` compatibility

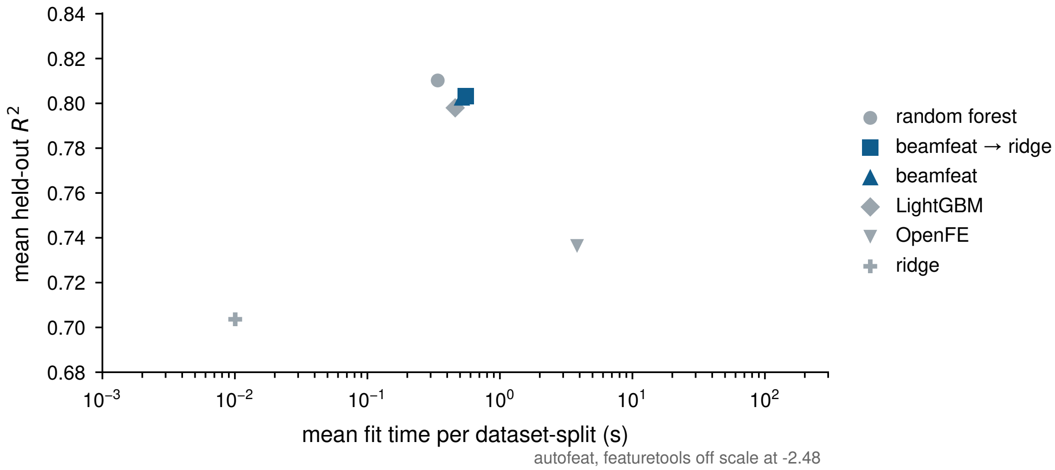

Figure 4: Mean held-out R^2 against mean fit time across the 360-fit study, on a logarithmic time axis. No error bars: the spread across the 45 fits per method is dominated by differences between the nine datasets rather than by uncertainty about a method, so an interval drawn from it would invite the wrong reading. `featuretools` falls below the axis at -2.48.

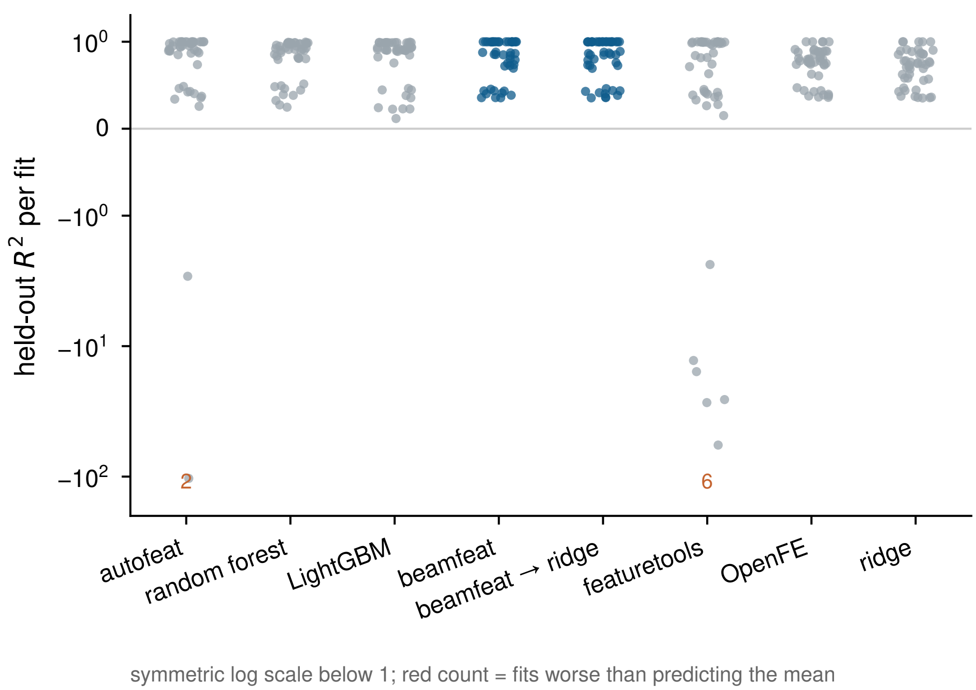

Figure 5: Every fit in the comparison study, one column per method, on a symmetric log scale below 1 so catastrophic results stay on the axis. A mean conceals what the tail does: `featuretools` and `autofeat` each produce fits far worse than predicting the target mean, while `beamfeat`'s poorest result across all 45 fits is R^2 0.355.

suite; tests that exercise 95% of the code; automatic re-testing of every change (continuous integration) against both the oldest supported and the newest dependency versions; tutorial notebooks that execute end to end; and every figure above produced by a script shipped in the repository.

The version described here is archived on Zenodo (29); the development repository is at https://github.com/LD-Shell/beamFeat, and the package installs from PyPI as `pip install beamfeat`.

Discussion and limitations

Three limits bound what the measurements above establish. The marginal null cannot see a feature that is jointly essential yet nearly independent of the target on its own; Friedman #1 prices that at roughly 0.086 of R^2, and methods that select jointly do better on such structure while certifying nothing. The comparison study spans nine datasets, which leaves the omnibus test underpowered: it separates no pair of methods, and the paired tests carry the inference. And every compared tool is run at default or paper-recommended settings with no hyperparameter search, a protocol that understates `OpenFE` in the gradient-boosted setting it was designed for and `featuretools` outside its relational use case.

Two directions follow. Set-level scoring would close part of the remaining search gap, but only by reintroducing the adaptivity the guarantee excludes; whether a selective-inference correction can recover that power without forfeiting error control is open. Second, the guarantee currently certifies association rather than predictive contribution, and the post-fit holdout check that separates them is a diagnostic, not a second guarantee.

Acknowledgements

The design of `beamfeat` was informed by a review of the `autofeat` source code (12), whose approach it builds on and departs from as described above. The author conceived the method, set its requirements and acceptance criteria, and directed development, using Anthropic's Claude as an AI assistant for implementation and drafting. Every figure and table in this paper is regenerated by a script that ships in the repository, and the full tests run automatically on every change. The one exception is noted where it arises: three of the four executions of the comparison study were superseded and their per-fit records not retained, so those means are reported from run transcripts rather than archived data. The author reviewed all code and text and takes sole responsibility for the work. This work received no external funding.

References

[1] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: http://dx.doi.org/10.1038/s41586-020-2649-2

[3] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, A. Vijaykumar, A. P. Bardelli, A. Rothberg, A. Hilboll,

A. Kloeckner, A. Scopatz, A. Lee, A. Rokem, C. N. Woods, C. Fulton, C. Masson, C. Häggström, C. Fitzgerald, D. A. Nicholson, D. R. Hagen, D. V. Pasechnik, E. Olivetti, E. Martin, E. Wieser, F. Silva, F. Lenders, F. Wilhelm, G. Young, G. A. Price, G.-L. Ingold, G. E. Allen, G. R. Lee, H. Audren, I. Probst, J. P. Dietrich, J. Silterra, J. T. Webber, J. Slavič, J. Nothman, J. Buchner, J. Kulick, J. L. Schönberger, J. V. de Miranda Cardoso, J. Reimer, J. Harrington, J. L. C. Rodríguez, J. Nunez-Iglesias, J. Kuczynski, K. Tritz, M. Thoma, M. Newville, M. Kümmerer, M. Bolingbroke, M. Tartre, M. Pak, N. J. Smith, N. Nowaczyk, N. Shebanov, O. Pavlyk, P. A. Brodtkorb, P. Lee, R. T. McGibbon, R. Feldbauer, S. Lewis, S. Tygier, S. Sievert, S. Vigna, S. Peterson, S. More, T. Pudlik, T. Oshima, T. J. Pingel, T. P. Robitaille, T. Spura, T. R. Jones, T. Cera, T. Leslie, T. Zito, T. Krauss, U. Upadhyay, Y. O. Halchenko, and Y. Vázquez-Baeza, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Feb. 2020. [Online]. Available: http://dx.doi.org/10.1038/s41592-019-0686-2

[4] J. Taylor and R. J. Tibshirani, "Statistical learning and selective inference," *Proceedings of the National Academy of Sciences*, vol. 112, no. 25, pp. 7629–7634, Jun. 2015. [Online]. Available: http://dx.doi.org/10.1073/pnas.1507583112

[5] L. Wasserman and K. Roeder, "High-dimensional variable selection," *The Annals of Statistics*, vol. 37, no. 5A, Oct. 2009. [Online]. Available: http://dx.doi.org/10.1214/08-AOS646

[6] B. Phipson and G. K. Smyth, "Permutation p-values should never be zero: Calculating exact p-values when permutations are randomly drawn," *Statistical Applications in Genetics and Molecular Biology*, vol. 9, no. 1, 2010, article 39.

[7] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.

[8] Y. Benjamini and D. Yekutieli, "The control of the false discovery rate in multiple testing under dependency," *The Annals of Statistics*, vol. 29, no. 4, pp. 1165–1188, 2001.

[9] R. F. Barber and E. J. Candès, "Controlling the false discovery rate via knockoffs," *The Annals of Statistics*, vol. 43, no. 5, pp. 2055–2085, 2015.

[10] E. Candès, Y. Fan, L. Janson, and J. Lv, "Panning for gold: 'model-x' knockoffs for high dimensional controlled variable selection," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 80, no. 3, pp. 551–577, 2018.

[11] D. R. Cox, "A note on data-splitting for the evaluation of significance levels," *Biometrika*, vol. 62, no. 2, pp. 441–444, Aug. 1975. [Online]. Available: http://dx.doi.org/10.1093/biomet/62.2.441

[12] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Machine Learning and Knowledge Discovery in Databases: International Workshops of ECML PKDD 2019, Proceedings, Part I*, ser. Communications in Computer and Information Science, vol. 1167. Springer, 2020, pp. 111–120.

[13] T. Zhang, Z. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and J. Li, "OpenFE: Automated feature generation with expert-level performance," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 41 880–41 901.

[14] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *2015 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.

[15] A. Spector and L. Janson, "Powerful knockoffs via minimizing reconstructability," *The Annals of Statistics*, vol. 50, no. 1, Feb. 2022. [Online]. Available: http://dx.doi.org/10.1214/21-AOS2104

[16] M. Cranmer, "Interpretable machine learning for science with PySR and SymbolicRegression.jl," *arXiv preprint arXiv:2305.01582*, 2023.

[17] M. Christ, N. Braun, J. Neuffer, and A. W. Kempa-Liehr, "Time series feature extraction on basis of scalable hypothesis tests (tsfresh – a Python package)," *Neurocomputing*, vol. 307, pp. 72–77, 2018.

[18] J. H. Friedman, "Multivariate adaptive regression splines," *The Annals of Statistics*, vol. 19, no. 1, Mar. 1991. [Online]. Available: http://dx.doi.org/10.1214/aos/1176347963

[19] J. D. Storey, "A direct approach to false discovery rates," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 64, no. 3, pp. 479–498, Aug. 2002. [Online]. Available: http://dx.doi.org/10.1111/1467-9868.00346

[20] W. La Cava, P. Orzechowski, B. Burlacu, F. O. de França, M. Virgolin, Y. Jin, M. Kommenda, and J. H. Moore, "Contemporary symbolic regression methods and their relative performance," 2021. [Online]. Available: https://arxiv.org/abs/2107.14351

[21] S.-M. Udrescu and M. Tegmark, "AI Feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, Apr. 2020. [Online]. Available: http://dx.doi.org/10.1126/sciadv.aay2631

[22] K. B. Gorman, T. D. Williams, and W. R. Fraser, "Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus *Pygoscelis*)," *PLoS ONE*, vol. 9, no. 3, p. e90081, Mar. 2014. [Online]. Available: http://dx.doi.org/10.1371/journal.pone.0090081

[23] I.-C. Yeh, "Modeling of strength of high-performance concrete using artificial neural networks," *Cement and Concrete Research*, vol. 28, no. 12, pp. 1797–1808, Dec. 1998. [Online]. Available: http://dx.doi.org/10.1016/S0008-8846(98)00165-3

[24] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision Support Systems*, vol. 47, no. 4, pp. 547–553, Nov. 2009. [Online]. Available: http://dx.doi.org/10.1016/j.dss.2009.05.016

[25] D. Harrison and D. L. Rubinfeld, "Hedonic housing prices and the demand for clean air," *Journal of Environmental Economics and Management*, vol. 5, no. 1, pp. 81–102, Mar. 1978. [Online]. Available: http://dx.doi.org/10.1016/0095-0696(78)90006-2

[26] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[27] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001. [Online]. Available: http://dx.doi.org/10.1023/A:1010933404324

[28] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*. Curran Associates, 2017, pp. 3146–3154.

[29] O. M. Daramola, "beamfeat: beam-search feature construction with false-discovery-rate controlled selection," 2026. [Online]. Available: https://doi.org/10.5281/zenodo.21677596